

Project Design Phase-II Technology Stack (Architecture & Stack)

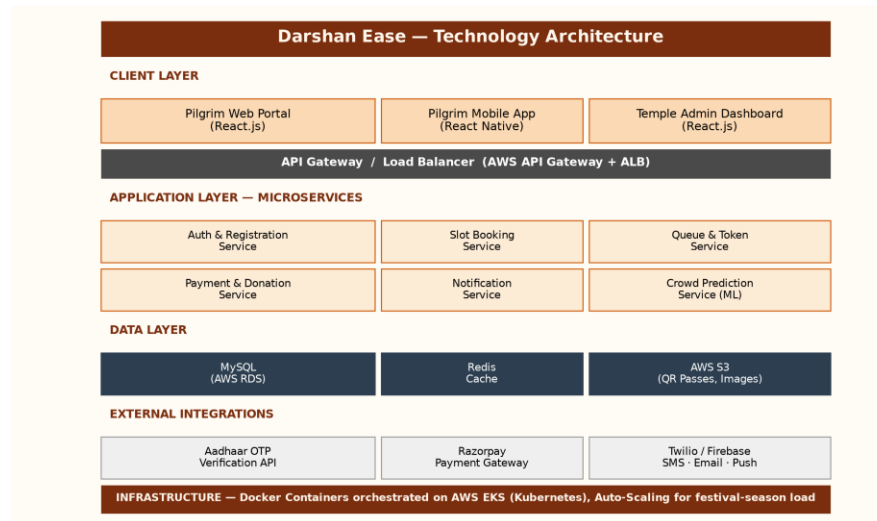
Date	15 July 2026
Team ID	SWTID-2026-7507
Project Name	Darshan Ease
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

Darshan Ease – Temple Darshan Slot Booking & Queue Management: System Architecture

Reference: <https://c4model.com/>



Guidelines:

- Include all the processes (As an application logic / Technology Block)
- Provide infrastructural demarcation (Local / Cloud)
- Indicate external interfaces (third party API's etc.)
- Indicate Data Storage components / services
- Indicate interface to machine learning models (if applicable)

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1	User Interface	Web portal and mobile app where pilgrims search temples, check live queue status, and book darshan slots	React.js (Web), React Native (Mobile), HTML5, CSS3
2	Application Logic-1 (Slot Booking)	Handles darshan slot search, availability check, and booking logic	Node.js, Express.js
3	Application Logic-2 (Queue Management)	Generates real-time tokens and manages live queue / crowd status updates	Node.js with Socket.io
4	Application Logic-3 (Notifications)	Sends booking confirmations, queue-turn alerts, and reminders	Firebase Cloud Messaging, Twilio SMS/Email API
5	Database	Stores pilgrim profiles, bookings, temple and slot data	MySQL
6	Cloud Database	Managed relational database hosted on cloud	AWS RDS (MySQL)
7	File Storage	Stores QR e-passes, temple images, and ID proof uploads	AWS S3
8	External API-1	Processes booking fees, donations, and prasad orders	Razorpay Payment Gateway API
9	External API-2	Verifies pilgrim identity during registration	Aadhaar-based OTP Verification API
10	Machine Learning Model	Predicts crowd density and recommends optimal darshan slots	Crowd Prediction Model (Python, scikit-learn, time-series regression)
11	Infrastructure (Server / Cloud)	Local Server Configuration: Docker containers for development Cloud Server Configuration: AWS EC2 + Kubernetes (EKS) with auto-scaling for festival traffic	AWS Cloud, Kubernetes, Docker

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	Open-source frameworks used to build the application	React.js, Node.js, Express.js, Redis
2	Security Implementations	Access controls, encryption and identity checks implemented	JWT Authentication, HTTPS/TLS, AES-256 Encryption, Aadhaar OTP Verification, OWASP-compliant coding practices
3	Scalable Architecture	Justification of scalability for handling festival-season traffic spikes	Microservices architecture containerized with Docker & orchestrated via Kubernetes (horizontal auto-scaling)
4	Availability	Justification of high availability during peak pilgrim traffic	Multi-AZ deployment on AWS with Application Load Balancer and auto-scaling groups (99.9% uptime SLA)
5	Performance	Design considerations for handling high concurrent booking requests	Redis caching for live queue data, CloudFront CDN for static assets, supports 10,000+ concurrent bookings

References:

<https://c4model.com/>

<https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/>

<https://www.ibm.com/cloud/architecture>

<https://aws.amazon.com/architecture>

<https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d>